

About

A user-friendly, lightweight TUI for disk imaging

#rust #tui #disk-image

- Readme
- GPL-3.0 license
- Contributing
- Activity
- 1.9k stars
- 12 watching
- 27 forks

Report repository

Releases 13

v0.4.10

Latest

on Oct 20, 2025

+ 12 releases

Packages

No packages published

Contributors 9



Languages

Rust 84.7%

Nix 7.6%

Python 4.1%

Objective-C 1.9%

C 0.8%

Shell 0.8%

Other 0.1%



971b18a · 3 months ago



.github

Auto-generate --help output in R...

5 months ago



.vscode

Make VSCode extension recom...

3 years ago



checks

Wrap UI tests into a NixOS test

5 months ago



images

Update header image

3 months ago



native/darwin

Make enumdisk objc instead of ...

3 years ago



nix

2025-10-13 updates (#198)

5 months ago



packaging/aur

do AUR stuff in CI

3 years ago



scripts

Auto-generate --help output in R...

5 months ago



src

Fix unused import

3 months ago



.envrc

Initial commit

3 years ago



.gitignore

chore: ignore IntelliJ folder

5 months ago



CONTRIBUTING.md

Update CONTRIBUTING.md (#...

2 years ago



Cargo.lock

feat(ui): add estimated completi...

5 months ago



Cargo.toml

feat(ui): add estimated completi...

5 months ago



LICENSE

Make a README and add a LI...

3 years ago



README.md

Update header image

3 months ago



build.rs

upgrade buildtime deps

5 months ago



default.nix

Initial commit

3 years ago



flake.lock

2025-10-13 updates (#198)

5 months ago



flake.nix

Auto-generate --help output in R...

5 months ago



shell.nix

Initial commit

3 years ago



README



Contributing

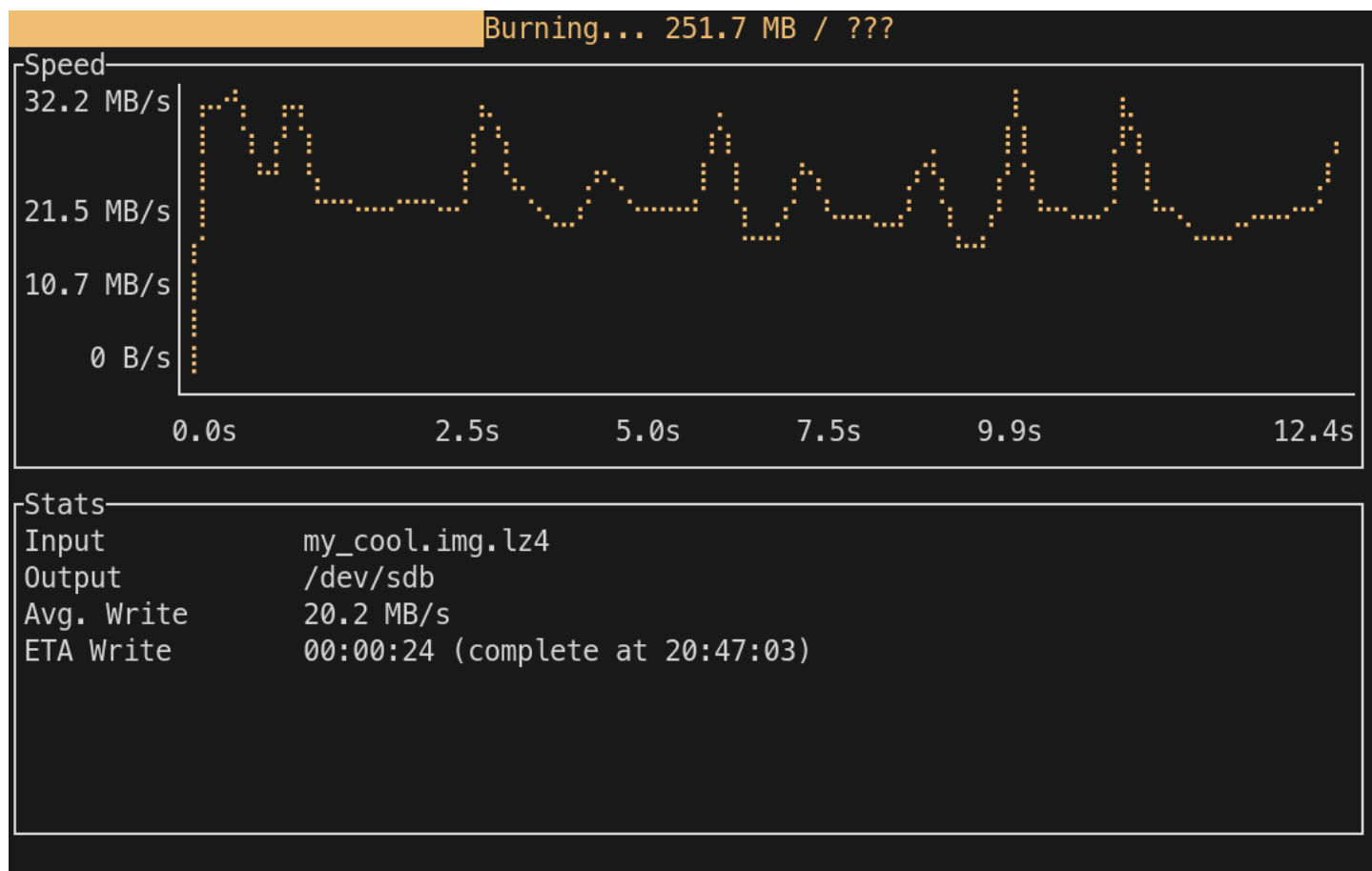


GPL-3.0 license



Caligula Burning Tool

CI failing



Caligula is a user-friendly, lightweight TUI for imaging disks.

```
$ caligula
```

```
A lightweight, user-friendly disk imaging tool
```

```
Usage: caligula
```

```
    caligula burn [OPTIONS] <IMAGE>
```

```
    caligula help [COMMAND]...
```

```
Options:
```

```
  -h, --help      Print help
```

```
  -V, --version   Print version
```

```
caligula burn:
```

```
A lightweight, user-friendly disk imaging tool
```

```
  <IMAGE>          Input image to burn
```

```
  -o <OUT>         Where to write the output. If not supplied,  
we will search for possible disks and ask you for where you want to burn
```

```
  -z, --compression <COMPRESSION> What compression format the input file is  
in [default: ask] [possible values: ask, auto, none, gz, bz2, xz, lz4, zst]
```

```
  -s, --hash <HASH> The hash of the input file. For more  
information, see long help (--help) [default: ask]
```



```

--hash-file <HASH_FILE>      Where to look for the hash of the input
file
--hash-of <HASH_OF>          Is the hash calculated from the raw file,
or the compressed file? [possible values: raw, compressed]
--show-all-disks            If provided, we will show all disks,
removable or not
--interactive <INTERACTIVE>  If we should run in interactive mode or not
[default: auto] [possible values: auto, always, never]
-f, --force                  If supplied, we will not ask for
confirmation before destroying your disk
--root <ROOT>                If we don't have permissions on the output
file, should we try to become root? [default: ask] [possible values: ask,
always, never]
-h, --help                  Print help (see more with '--help')
-V, --version                Print version

```

caligula help:

Print this message or the help of the given subcommand(s)

[COMMAND]... Print help for the subcommand(s)

Features

- **Cool graphs** that show you how fast you're writing
- **Listing attached disks**, and telling you their size and hardware model information
- **Decompressing** your input file for a variety of formats, including gz, bz2, and xz
- **Validating your input file against a hash before burning**, with support for md5, sha1, sha256, and more!
- **Running sudo/doas/su** if you forgot to run as `root` earlier (it happens)
- **Rich confirmation dialogs** so you don't accidentally nuke your filesystem
- **Verifying your disk after writing** to make sure it was written correctly
- **Small binary size** of <5 megabytes, even when statically linked
- Did I mention *cool graphs*?

How to install

There are a couple of ways to install Caligula.

- **Binary release:** You can download pre-built binaries from [the latest Github release](#).
- **Arch Linux:**
 - [Official repository](#): `pacman -S caligula`
 - [caligula-bin on the AUR](#): We also automatically publish binaries with every release.
 - [caligula-git on the AUR](#): Build from latest commit on `main` branch
 - [caligula-git on archlinuxcn](#): Prebuilt binaries from latest commit on `main` branch
- **Nix:**

- [Nixpkgs](#): `nix-env -i caligula`
- Repository flake: If your system is flake-enabled, you can use `github:ifd3f/caligula` for bleeding-edge changes.
- **Homebrew**: [philocalyst has made a homebrew tap for caligula](#): `brew tap philocalyst/tap && brew install caligula`
- **Cargo**: Caligula is published on [crates.io](#). Just run `cargo install caligula`
- **Build from source**: This is a relatively standard cargo project so you should be able to just `git clone` and `cargo build --release` it.

Platform support matrix

OS	Architecture	Automated tests	Automated builds	Published binaries
Linux	x86_64	✓	✓	✓
	aarch64	✗	✓	✓
MacOS	x86_64	✓	✓	✓
	aarch64	✓	✓	✓

Linux for other architectures theoretically works, but we are not making any guarantees.

We plan on supporting Windows, FreeBSD, and OpenBSD Eventually™. If you would like support for other OSes and architectures, please file an issue!

FAQ

Why did you make this?

Because I had to image one too many USB drives and wanted a nice, user-friendly `dd` alternative that wasn't 90 95 **413 MB**.

No really. A certain other tool doing the same exact thing has ballooned to 413 MB now.

```
% unzip balenaEtcher-linux-x64-2.1.4.zip
% du -sh balenaEtcher-linux-x64
413M    balenaEtcher-linux-x64
```



Why is it called "Caligula"?

There used to be a tool called Nero Burning ROM, so I chose another crazy Roman emperor to name this software after. It's a very uncreative name. I was originally planning on changing it later, but it's stuck.

Why is dd not good enough for you? Do you not know how to use it?

I know how dd works. In fact, to prove it, I have written a tutorial on it here, without using any AI.

How most people use dd to write ISOs to a USB drive or SD card

First, take a hash of your file, just to make sure that it didn't get corrupted in transit or bitrotted while living on your disk, because that happens occasionally. Most people forget to do this step, or are too lazy to. Most tutorials neglect to mention this step as well.

```
$ sha256sum some-image-file.iso.gz
```



After you have verified that the hashes match, you can finally unzip your file, if it came zipped.

```
$ gunzip some-image-file.iso.gz
```



Once that's done, you can finally start typing out your dd command. Using bs=4M because someone online recommended it and you forgot why, you type:

```
$ dd bs=4M if=some-image-file.iso of=/dev/
```



You get to this point in typing it out before realizing you forgot to consult lsblk !

```
$ lsblk
```



```
NAME            MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda              8:0    0   1.8T  0 disk
└─sda1           8:1    0   1.8T  0 part
sdb              8:16    1   3.6G  0 disk
└─sdb1           8:17    1   3.6G  0 part
sdc              8:32    1    60G  0 disk
nvme0n1          259:0    0 931.5G  0 disk
├─nvme0n1p1      259:1    0   550M  0 part /boot
└─nvme0n1p2      259:2    0   931G  0 part
```



You probably want the disk here either called **sdc**. Make sure you *don't confuse it with any of the other disks plugged into your computer, removable or otherwise*. That would be bad, because you might overwrite important data.

```
$ dd bs=4M if=some-image-file.iso of=/dev/sdc
```



Pause here *one more time*, cross referencing what you typed in with the output of `lsblk`, to *double-confirm* that you are indeed typing in the **correct disk** and not the **wrong disk** so that you don't **nuke any important disks storing important data such as your /home or your OS**.

Then, open a new terminal and type

```
$ lsblk
```



again to **triple-confirm** that you are indeed typing in the **correct disk** and not the **wrong disk** so that you don't **nuke any important disks storing important data such as your /home or your OS**. Doing so would **really ruin your day**.

Once you have **quadruple-confirmed** that you are indeed typing in the **correct disk** and not the **wrong disk** so that you don't **nuke any important disks storing important data such as your /home or your OS**, you can hit the enter key and finally actually run `dd`.

```
dd: failed to open '/dev/sdc': Permission denied
```



Of course, you probably forgot to type `sudo`. Make sure you do that.

```
$ sudo dd bs=4M if=some-image-file.iso of=/dev/sdc
```



By default, `dd` does not have any output. If you want to see the progress, you will need to cancel the command and add `status=progress` on.

```
^C^C^C
$ sudo dd bs=4M if=some-image-file.iso of=/dev/sdc status=progress
2684354560 bytes (2.7 GB, 2.5 GiB) copied, 1 s, 2.7 GB/s
```



Now, it's finally written!

At this point, it's probably a good idea to verify that the disk was written correctly. I personally don't know the command to do that. Do you know the command to do that? If you ask most pro sysadmins this, could they name the command? If you look at any tutorial online, do they list the command? **The answer to all of these is no, because nobody bothers do this.**

While `dd` is nice for scripting, after doing this process manually hundreds of times with hundreds of files and tens of drives, why don't you try using a simpler, more user-friendly process?

How most people use `caligula` to write ISOs to a USB drive or SD card

Typically, you would run

```
$ caligula burn some-image-file.iso.gz
```



and follow prompts in the terminal to allow the computer to fill in the blanks. In general, computers are good at filling in blanks. In fact, that was a big part of why we invented them.

Why Rust?

Because it's 🚀🚀🚀 BLAZING FAST 🚀🚀🚀 and 💾💾💾 MEMORY SAFE 💾💾💾 and 🦀🦀🦀 CRAB 🦀🦀🦀

On a serious note, I just like the language.

Why Nix?

It makes the CI more predictable.

Why so many other dependencies?

To be fair, Rust doesn't have a very comprehensive standard library, and I only use one or two functions in most of those dependencies. Thanks to dead code elimination, inlining, and other optimizations, they don't end up contributing much to the overall binary size.

Why do you have to type in `burn` ? Will you add other subcommands later?

Yes. I Eventually™ plan on adding other capabilities, like [Windows install disk support](#) and [secure disk erasure](#), and those will end up in their own subcommands.